

Frontier Explorer Node

Scoring · Penalties · Bonuses · Blacklist System — Complete Reference

Ros_lidar_bot | Branch: device-amey

1. Overview

The frontier explorer node autonomously directs the robot to unexplored areas (frontiers — unknown cells adjacent to free cells in the SLAM occupancy grid). Every 3 seconds it scores all available frontiers, sends the best one to Nav2, and maintains a standby queue for instant switching when the current goal fails. A multi-tier blacklist prevents the robot from repeatedly visiting unreachable zones.

2. Configuration Parameters (frontier_explorer.yaml)

Parameter	Value	Description
min_frontier_size	3 cells	Minimum cluster size to consider (relaxed progressively when no frontier found)
min_goal_distance	0.35 m	Ignore frontiers closer than this — robot is already near them
max_goal_distance	5.5 m	Normal search radius; auto-scales to 50% of room's larger dimension on startup
max_goal_distance_cap	12.0 m	Absolute ceiling — never send goal further than this even after relaxation
max_range_expansion_factor	2.0×	If nothing in normal range, search up to 2× the range as a single fallback
wall_safe_distance	0.15 m	Reject frontier targets within 3 map-cells of the map boundary (prevents worldToMap errors)
goal_blacklist_radius	0.80 m	Standard exclusion zone around a failed position
blacklist_timeout_sec	300 s	Standard blacklist duration (5 minutes)
size_weight	1.8	Score bonus for larger frontier clusters (more unexplored area revealed)
distance_weight	1.0	Score penalty for distance to frontier
heading_weight	0.5	Score penalty for heading error (robot must turn to face frontier)
openness_weight	0.7	Score bonus for open space around the frontier target cell
improvement_margin	0.20	New goal must beat active goal score by this margin to trigger a switch
goal_interval_sec	3.0 s	How often the explorer ticks (checks for new frontiers)
goal_cooldown_sec	3.0 s	Pause after a goal ends before selecting the next one
goal_reached_radius	0.45 m	Mark active goal as reached when robot comes within this distance
goal_timeout_sec	90.0 s	Hard cancel if navigating to goal takes longer than this
progress_timeout_sec	25.0 s	Cancel if robot does not move ≥ 12 cm in this time window
no_frontier_done_ticks	20 (auto)	Consecutive no-frontier ticks before declaring exploration complete

3. Frontier Detection Pipeline

3.1 Enclosed-Region Elimination (pre-filter)

Before any frontier is extracted the map undergoes a flood-fill from its boundary edges through all traversable (free OR unknown) cells. Any unknown cell that cannot be reached is completely enclosed by occupied cells — permanently occluded (e.g. the interior of a sofa, void behind solid bookshelf panels). These cells are virtually marked as occupied so no frontier ever forms there.

```
traversable = free | unknown
reachable   = BFS from map boundary through traversable
enclosed    = unknown & ~reachable -> mark as occupied (100)
```

3.2 Frontier Cell Extraction (numpy vectorised)

After enclosed-region removal, frontier cells are found with 8-connectivity: an unknown cell is a frontier if any of its 8 neighbours is free. A numpy roll-based mask makes this $O(W \times H)$ without Python loops.

3.3 BFS Cluster Grouping

Frontier cells are grouped into connected clusters using 8-connectivity BFS. Each cluster is one candidate frontier for scoring.

3.4 Per-Cluster Filters (before scoring)

Filter	Condition	Effect
Size filter	<code>len(cluster) < min_frontier_size</code>	Skip — too small to be worth a dedicated trip
Target exists	No free cell adjacent to the cluster	Skip — cluster touches no navigable space
Map boundary guard	Target within <code>wall_safe_distance</code> (0.15 m = 3 cells) of map edge	Skip — causes worldToMap failures
Blacklist check	Target within blacklist radius of any failed position	Skip — known bad area
Distance filter	<code>dist < min_goal_distance</code> OR <code>dist > max_goal_distance</code>	Skip — too close or too far
Target clearance floor	Openness of target cell < 0.15 (fewer than 15% free neighbours in 3-cell radius)	Skip — robot would be wedged against an obstacle to reach it

4. Scoring Formula

4.1 Base Score

Every frontier that passes all filters is scored with:

```
score = (size_weight    x size_term)
        - (distance_weight x distance_term)
        - (heading_weight x heading_term)
        + (openness_weight x openness_term)
```

Term	Formula	Range	Weight	Direction
<code>size_term</code>	<code>len(cluster) / max_cluster_size_in_current_scan</code>	0 – 1	1.8	Higher = BETTER (more unexplored area revealed per trip)
<code>distance_term</code>	<code>distance_to_target / max_goal_distance</code>	0 – 1	1.0	Higher = WORSE (farther away costs more)

Term	Formula	Range	Weight	Direction
heading_term	$\text{abs}(\text{normalise}(\text{angle_to_target} - \text{robot_yaw})) / \pi$	0 – 1	0.5	Higher = WORSE (requires a bigger turn before moving)
openness_term	$\text{free_cells} / \text{total_cells}$ in 3-cell radius around the target cell	0 – 1	0.7	Higher = BETTER (open space, safe approach route)

4.2 Far-Frontier Bonus (activates when ALL frontiers are far)

When the closest available frontier exceeds 35% of `max_goal_distance` the explorer enters bonus mode. The distance penalty is reduced proportionally to frontier size so large unexplored zones beat small far-away clusters:

```
near_zone = max_goal_distance x 0.35
IF min_dist_available > near_zone:      # all frontiers are far
    bonus = min(0.40, size_ratio x 0.45)
    distance_term = max(0.0, dist/max_dist - bonus)
```

A large far frontier (`size_ratio` = 1.0) earns a 0.40 bonus — its distance term is almost halved. A small far frontier (`size_ratio` = 0.1) earns only 0.045. This prevents exploration from stalling when only distant zones remain.

4.3 Hysteresis Guard

While a goal is active, a new candidate must beat it by `improvement_margin` (0.20) to trigger a switch. This prevents constant goal-switching for marginally better options.

```
IF active_goal is set AND new_score < active_score + 0.20:
    keep current goal (no switch)
```

5. Blacklist System — Getting IN and OUT

5.1 Four Tiers of Blacklisting

Tier	Trigger	Radius	Duration	Log message
Standard	STATUS_CANCELLED — our timeout or safety switch fired	0.80 m	300 s	No progress — blacklisting...
Repeat failure (2nd+ time)	Same area blacklisted again within 1.2 m merge radius	1.60 m (doubled)	300 s	Area blacklisted Nx — effective radius now 1.60 m
Dead Zone — Nav2 aborted	STATUS_ABORTED — Nav2 itself gave up; no path found or controller failed	1.60 m (immediate)	600 s	DEAD ZONE marked — Nav2 aborted (unreachable)
Dead Zone — Phantom success	Robot 'reached' same frontier 3 times without clearing the unknown cells	1.60 m (immediate)	600 s	Permanently occluded area — blacklisted 10 min

5.2 How Blacklist Radius Doubles (Adaptive Radius)

Every call to `_blacklist_goal()` first looks for an existing blacklist entry within 1.2 m (1.5× base radius) of the new position. If found it increments the hit counter for that entry. `_is_blacklisted()` then uses double radius for any entry with `hits > 1`:

```
_is_blacklisted(x, y):
    FOR each (bx, by) in failed_goals:
        hits = _blacklist_hits[(bx,by)]    # default 1
        r    = 1.60 m IF hits > 1 ELSE 0.80 m
```

```
IF distance(x,y, bx,by) <= r: return TRUE
```

5.3 Minimum Distance from Last Cancellation Point

After any cancellation `_last_cancel_xy` is recorded. When the standby queue is polled for the next frontier, any candidate within `goal_blacklist_radius × 2.0` (= 1.60 m) of `_last_cancel_xy` is skipped. This stops the robot from immediately re-selecting a frontier that is 0.85 m away from where it just got stuck — just outside the standard blacklist radius.

5.4 Getting OUT of the Blacklist

Mechanism	How the blacklist entry is removed
Standard timeout	After 300 s the entry is pruned by <code>_prune_failed_goals()</code> at the start of each tick. The hit counter for that position is also deleted.
Dead zone timeout	After 600 s (10 min) the entry is pruned. Hit counter deleted. The area becomes available again from scratch.
Genuine success clears phantom count	If <code>STATUS_SUCCEEDED</code> AND <code>_frontier_vanished()</code> returns True (cells cleared), the phantom counter for that position is reset to 0. No blacklist is applied.
<code>_last_cancel_xy</code> cleared on success	When any goal SUCCEEDS the minimum-distance restriction is lifted: <code>_last_cancel_xy</code> = None. The next queued frontier can be anywhere.
No manual early-exit	There is no mechanism to remove a blacklist entry before its timeout. The robot must wait. Raising <code>blacklist_timeout_sec</code> makes areas stay excluded longer.

6. Frontier Queue — Pre-computed Standby List

6.1 What Goes In

Every time a new goal is sent, `_score_frontiers` runs with `collect_all=True`. After the best frontier is dispatched, remaining valid frontiers build the queue:

Rule	Detail
Large frontiers only	Cluster must have $\geq \max(\text{min_frontier_size} \times 3, 15)$ cells to enter the queue. Small clusters get mapped naturally as the robot passes nearby — no dedicated trip needed.
Geographic diversity	Each new queue entry must be $\geq \text{goal_blacklist_radius} \times 2.5$ (= 2.0 m) from every already-queued entry. Ensures the robot has options in different parts of the room.
Fallback	If diversity pruning leaves fewer than <code>queue_size</code> (5) entries, fill remaining slots with any unqueued valid frontier regardless of size.
Queue size	Maximum 5 entries. Stale after 20 s — fully discarded, triggers fresh rescan.

6.2 Validation Before Use (lazy evaluation)

Queue entries are re-validated at the moment of use — not when built:

```
_pop_queued_frontier(robot_x, robot_y):
    IF queue empty OR age > 20 s:                return None
    FOR each (xy, score) in queue:
        IF _is_blacklisted(xy):                  skip
        IF _frontier_vanished(xy):                skip (mapped while waiting)
        IF dist < min_goal_distance:              skip
        IF dist > max_goal_distance_cap:          skip
```

```

IF dist_from_last_cancel < 1.60 m:    skip  (too close to stuck point)
re-score from CURRENT robot position
return (xy, fresh_score)

```

6.3 When the Queue Is Used

Trigger	Delay	Action
Safety stop blocked	3 s of continuous blocking	Pop queue, blacklist current goal, send next frontier immediately
Progress timeout	8 s without 12 cm movement	Pop queue, blacklist current goal, send next in the same tick (no wait for next cycle)
Frontier already mapped	Detected on next tick	Cancel goal without blacklisting, revert to full frontier rescan
Queue stale or empty	20 s since queue was built	Queue is cleared — full fresh frontier scan runs on the next tick

7. Progressive Relaxation (when no frontier found)

Each tick that returns no valid frontier increments `_no_frontier_ticks`. Every 3 ticks a constraint is loosened. Relaxation is sticky — parameters stay loose for the rest of the session.

Tick	What changes	Effect
Tick 3	<code>min_frontier_size</code> reduced by 1	Accepts smaller clusters previously filtered as too small
Tick 6	<code>min_frontier_size</code> reduced by 1 + <code>max_goal_distance</code> extended to cap (12 m)	Opens search to the full room range
Tick 9	<code>min_frontier_size</code> reduced by 1 + <code>min_goal_distance</code> halved	Accepts frontiers very close to the robot
Tick 12+	<code>min_frontier_size</code> reduced by 1 every 3 ticks until floor (1)	Accepts any single unknown cell as a frontier
<code>>= done_ticks</code>	Exploration declared complete	Node logs 'exploration complete', stops sending goals

7.1 Auto-calibration from Map Size (one-time, on first map)

Two parameters are scaled to the actual room size:

```

max_goal_distance      = min(map_larger_dim x 0.50,  12.0 m)
no_frontier_done_ticks = max(config_value,  map_larger_dim x 2 ticks/metre)

```

```

Examples:  10 m room  ->  5.0 m range,  20 ticks
           15 m room  ->  7.5 m range,  30 ticks
           25 m hall  -> 12.0 m range,  50 ticks

```

8. Complete Goal Lifecycle — All Outcomes

Event	Blacklisted?	Queue used?	<code>_last_cancel_xy</code>	Phantom count
Goal sent to Nav2	No	Queue built	No change	No change
STATUS_SUCCEEDED + frontier cleared	No	Queue kept	Cleared (None)	Reset to 0

Event	Blacklisted?	Queue used?	_last_cancel_xy	Phantom count
STATUS_SUCCEEDED + frontier NOT cleared	After 3rd time: YES (10 min, 1.6 m)	Queue kept	Cleared (None)	Incremented
STATUS_ABORTED (Nav2 gave up)	YES immediately (10 min, 1.6 m)	Pop next	Set to failed pos	No change
STATUS_CANCELLED (our timeout / safety)	YES (5 min, 0.8 m)	Pop next	Set to failed pos	No change
Safety stop blocked 3 s	YES (5 min, 0.8 m)	Pop next	Set to failed pos	No change
Progress timeout (8 s no movement)	YES (5 min, 0.8 m)	Pop next same tick	Set to failed pos	No change
Goal timeout (90 s hard limit)	YES (5 min, 0.8 m)	Next tick full scan	Set to failed pos	No change
Frontier mapped mid-trip	No (clean cancel)	Next tick full scan	Cleared (None)	No change

9. Safety Stop Node Integration

safety_stop_node publishes /safety_blocked (Bool) on every laser scan (20 Hz). The explorer subscribes and tracks how long continuous blocking lasts.

Duration blocked	Explorer action
0 – 3 s	No action. Nav2 recovery behaviors (spin/backup) may activate internally to try to clear the path.
Reaches 3 s (continuous)	Explorer blacklists the current goal position, immediately pops the standby queue, and sends the next frontier without waiting.
Safety stop clears	_safety_blocked_since is reset to None. If blocking starts again, the 3 s countdown restarts from zero.

Angular velocity (angular.z) always passes through the safety stop unchanged so Nav2 spin recovery can rotate the robot even when forward motion is blocked.

10. Phantom Success — Permanent Occlusion Detection

A phantom success occurs when Nav2 returns STATUS_SUCCEEDED (robot within 0.30 m of target) but the unknown cells near the target have NOT cleared. This means the lidar cannot see into that zone from any accessible robot position — it is a permanently occluded area (e.g. void inside solid furniture).

```

ON STATUS_SUCCEEDED:
  IF _frontier_vanished(pos) == False:    # frontier still there
    _phantom_successes[pos] += 1
    log 'Frontier reached but not cleared (N/3)'
    IF count >= 3:
      blacklist_timeout = 600 s           # 10 minutes
      _blacklist_goal(pos)
      _blacklist_hits[pos] = 10           # force double radius
      log 'Permanently occluded – blacklisted 10 min'
  ELSE:
    _phantom_successes[pos] = 0           # genuine success, reset

```

11. Worked Scoring Example

Robot at (0, 0) facing East (yaw = 0). max_goal_distance = 5.5 m. All four frontiers pass the pre-filters. Frontiers B and D are in the same quadrant so only one will enter the diverse queue.

Frontier	Size	Distance	Heading Δ	Openness	Base Score	Outcome
A — open far corner	80 cells (ratio 1.0)	4.0 m	10°	0.75	$1.8 \times 1.0 - 1.0 \times 0.73 - 0.5 \times 0.06 + 0.7 \times 0.75 = 1.82$	BEST — sent as active goal
B — near sofa gap	30 cells (ratio 0.38)	1.5 m	45°	0.30	$1.8 \times 0.38 - 1.0 \times 0.27 - 0.5 \times 0.25 + 0.7 \times 0.30 = 0.68$	2nd — enters diverse queue
C — beside bookshelf	15 cells (ratio 0.19)	2.0 m	90°	0.10	openness 0.10 < 0.15 floor — SKIPPED	SKIPPED (clearance filter)
D — far wall (all far!)	60 cells (ratio 0.75)	5.2 m	15°	0.65	far bonus 0.34 applied: dist_term $0.95 \rightarrow 0.61 \rightarrow 1.77$	3rd (bonus) — queue if geographically spread from B

Frontier A is sent. B and D (both ≥ 15 cells) are candidates for the diverse queue. If B and D are ≥ 2.0 m apart, both enter. Otherwise only B (higher score) enters. C is permanently rejected by the clearance floor and is never queued or scored further.